High Level Synthesis for FPGA:Bridging Software and Hardware

Ikramul Hassan Sazid
Electronics Engineering (B. Eng)
Hamm-Lippstadt University of Applied Sciences
Lippstadt, Germany
ikramul-hassan.sazid@stud.hshl.de

***Abstract*—With the increase in the complexity of designs of System-on-Chip (SoC), the move away from manual Register Transfer Level (RTL) coding is inevitable. Manual RTL coding is time consuming and does not scale well in today's era. To enhance productivity and lower the high learning curve, along with advancing capabilities of the SoC design, High Level Synthesis (HLS) technology has become increasingly important. HLS facilitates automated translation of software- like high level untimed specifications into cycle-accurate RTL form. The paper covers the evolution, mathematical basis, algorithmic complexities, and implementation of HLS for FPGA (FPGA) devices and highlights its capability of bridging the software to hardware gap at an acceptable QoR. Index Terms—High Level Synthesis, FPGA, RTL, System-on- Chip, System of Difference Constraints, Pipelining.**

***Index Terms*—High-Level Synthesis, FPGA, RTL, System-on-Chip, System of Difference Constraints, Pipelining.**

I. Introduction

In pursuit of building tomorrow's microchip and System-on-Chip (SoC) designs that will be the backbone of future technology, the design is getting exponentially complex and the need to transition away from manual Register Transfer Level (RTL) coding is dire, as manual RTL coding becomes time-consuming and very hard as the complexity increases. To improve productivity, high-level synthesis can offer a lot in reducing development time, reducing the steep learning curve for RTL, and advancing SoC capabilities.

A. Core Concept and Core Definition

As the increasing complexity of SoC design demands, HLS technology has been established as a vital technology that needs to be utilized. HLS can be defined as an automated design process which converts high-level and untimed or partially timed software specification language constructs into low-level, cycle-accurate RTL language constructs. The architects will have the ability to explore architectural trade-offs based on area, power, and performance through a common functionality model before designing particular hardware micro-architectures.

B. Output Definition

The final output by HLS consists of a cycle-accurate, optimized RTL netlist. Advanced HLS tools today are capable of producing cycle-accurate RTL output in standard HDL languages such as Verilog, VHDL, or SystemC [1]. The RTL modules generated by modern HLS are regarded as regular hardware elements that can be used in conventional simulation environments for functional verification purposes. HLS raises the abstraction level from low to high and thus enables engineers to explore micro-architecture designs quickly, ensuring they don't have to be bogged down by time-consuming RTL designs where they have to design state machines and data paths manually.

Identify applicable funding agency here. If none, delete this.

II. History, Previous Limitations, and Current Advancements of HLS

The evolution of High-Level Synthesis (HLS) for FPGAs spans a few decades, transitioning from early theoretical research into a production-ready state. The very first stage of HLS can be thought of as Generation 0 and 1 (1970s to early 1990s) where researchers laid the algorithmic foundations with academic projects like CMU-DA, but these tools were mostly ignored by an industry actively adopting manual RTL synthesis.

During Generation 2, which was in the mid-1990s to early 2000s, major Electronic Design Automation (EDA) developers introduced the first commercial HLS tools, but they failed to achieve industry-wide adoption. This failure was driven by some severe limitations. Early tools often required designers to use untimed behavioral Hardware Description Languages (HDLs) instead of familiar software languages, imposing a steep learning curve. On top of that, the early systems made simplistic, "technology-independent" assumptions that failed to exploit specific hardware features, which resulted in the HLS code being significantly behind in Quality of Results (QoR) compared to hand-written RTL code. Earlier HLS technology also lacked datapath synthesis. As a result, it left the huge task of system integration and memory management completely to the RTL engineers.

In the early 2000s, HLS could utilize C, C++, and SystemC which were widespread in the industry. This ushered in the tipping point for HLS to be adopted by software and algorithm engineers. This time phase is now known as Generation 3.

Right now, we can safely say that HLS is in Generation 4. It is characterized by Accelerator-Centric Synthesis (ACS).

ACS is able to integrate full system-level design flows by connecting custom IPs, embedded CPUs, and complex memory hierarchies.

All these decades of advancement have dissolved the old QoR issue of HLS. It is now capable of applying aggressive, bit-accurate code optimizations, Static Single Assignment (SSA) transformations, and advanced pointer analysis before hardware generation. The core scheduling problem has been solved by the System of Difference Constraints (SDC). By using totally unimodular constraint matrices, SDC can generate integral scheduling solutions in polynomial time, bypassing the exponential computational overhead of older heuristic searches.

Because the constraint matrix is fully unimodular, the SDC algorithm can give integral solutions for scheduling problems using polynomial-time computation without any computational difficulty associated with the previous heuristic searching method. Also, the new HLS algorithms employ "soft constraints" to smartly assign the operations to specific, prefabricated blocks in the FPGA architecture (such as Xilinx DSP48 blocks).

Ultimately, these algorithmic and architectural advancements have transformed HLS. While expert manual RTL design can occasionally achieve marginally higher raw clock frequencies or lower basic resource usage, modern HLS effectively bridges the historical QoR gap. Most importantly, it reduces overall development time and required lines of code by 50% to 75%, cementing its status as a vital methodology for rapid, scalable FPGA engineering.

III. Formal and Mathematical Description of High-Level Synthesis

Conversion from a High-Level Software Language (C, C++, SystemC) into a cycle accurate Register-Transfer Level (RTL) form is not just a matter of translation but rather a complex multi-objective optimization problem with strong constraints.

A. The Core Scheduling Problem

A. The Fundamental Scheduling Problem In the conventional software compilation, the program op- erations are executed sequentially under the control of the program counter of the general purpose processor. An FPGA, however, provides a huge amount of an uncom- mitted space for hardware logic (Look-Up Tables, Flip-Flop cells, DSP blocks, and Block RAMs). So, the HLS compi- ler should find out the particular clock cycle when each algorithm operation is performed along with the mapping of operations onto particular hardware resources. There should be strict adherence to two major limitations here: data de- pendencies (an operation cannot be started until the data for it is ready) and the limitations of the physical resources (a number of concurrently operating operations cannot exceed the number of available DSP blocks and memory ports on the target FPGA) [5]. In previous generations of the HLS compilers, there has been an effort made to address the issue of scheduling using the methods of ILP and heuristic algorithms such as the list scheduling algorithm. Previous ILP formulations made use of binary decision variables and were denoted as $x_{i,c}$, such that if operation i was assigned at time c, $x_{i,c} = 1$, and 0 otherwise. In order to encode the solution of n operations over m possible steps, $O(m \times n)$ binary variables were needed by the solver [1]. Even though this methodology mathematically provided an optimal solution, it was extremely inefficient in terms of scalability. With highly complex modern day System-on-Chip (SoC) designs having thousands of operations, the ILP state space grew unfeasibly large, resulting in compilation times taking several days, defeating the purpose of HLS.

B. The System of Difference Constraints (SDC)

In order to resolve the scaling issues that come with traditional ILP, the current cutting-edge HLS tools (including AMD's Vitis HLS or the previous name, AutoPilot) incorporate an extremely advanced mathematical model called the System of Difference Constraints (SDC) [1]. SDC transforms the scheduling problem in a revolutionary way by using the continuous definition of time.

Rather than having variables for each possible time of all operations, SDC employs only one variable, s_i, which corresponds to the start time of operation i. This results in having the number of dimensions equal to $O(n)$. In the framework of SDC, dependencies can be formulated as the difference of two start times. For example, when operation j depends on the output of operation i, where operation i requires d cycles to finish, the corresponding constraint can be expressed as:

$$s_j - s_i \geq d \tag{1}$$

which is reformulated for the solver as:

$$s_i - s_j \leq -d \tag{2}$$

C. Formal Description of the HLS Algorithm

The HLS synthesis algorithm has a number of separate phases starting from the front-end parser. HLS software is usually implemented on the basis of strong compilers like LLVM which parse high-level C/C++ source code to a hardware-independent Intermediate Representation (IR). Next phase involves application of traditional software compiler algorithms to optimize Control-and-Datapath Flow Graph (CDFG). Particularly, Static Single Assignment (SSA) form is widely used in order to perform constant propagation, dead code elimination and loop-invariant code motion before any hardware mapping starts [1].

After that, optimization of the IR takes place and the SDC mathematical formulation is used to schedule hardware. In practice, however, hardware design can hardly be absolutely precise; there might be some preferences regarding pipeline stage, register bound, critical path etc., which are not requirements but just "soft" constrains. The SDC algorithm handles this problem very elegantly by including a violation variable (v_j) and a penalty function ($\phi_j(v_j)$) into the objective function. It means that the optimization solver will do global

optimization - breaking some soft constrains if this makes circuit dramatically faster or smaller.

The complete formal linear programming formulation for modern HLS scheduling is defined as:

Minimize:

$$c^T s + \sum_j \phi_j(v_j) \tag{3}$$

Subject to Hard Constraints:

$$Gs \leq p \tag{4}$$

Subject to Soft Constraints:

$$H_j s - v_j \leq q_j \quad \text{and} \quad -v_j \leq 0 \tag{5}$$

In this formulation, $c^T s$ represents the objective to minimize the overall latency (total clock cycles), $Gs \leq p$ represents the immutable physical resource limits of the FPGA and logical data dependencies, and the latter equations mathematically balance the soft constraints against the penalty of violating them [1].

IV. Algorithmic Runtime, Complexity, and Overhead

The adoption of the SDC mathematical model is not merely a theoretical exercise; it has profound implications for the practical usability of HLS tools by hardware engineers.

A. Algorithmic Complexity and Unimodularity

The most significant mathematical feature of SDC is the fact that the constraint matrix (G and H) is totally unimodular. This means that all the basic feasible solutions to the LP problem are integer vectors [1].

This is an absolutely enormous benefit in terms of FPGA synthesis. Since the operation of FPGAs is based on strictly defined clock cycles, the scheduling function needs to return integer values (an operation cannot be scheduled on cycle 3.5, but only on cycle 3). In standard ILP, making sure that the solver returns integer values is done using inefficient branch-and-bound approach, which is famous for being NP-hard. Since the SDC constraint matrix is totally unimodular, the HLS tool can simply use standard linear programming solvers. And thanks to the mathematics, the solver will automatically provide integral solution without branch-and-bound. Thus, the algorithm will have polynomial run time and the HLS tool will still be able to compile complex FPGA implementations efficiently [1].

B. Execution Overhead and Tool Trade-offs

However, despite all the efficiencies , HLS tools need to carefully maintain the balance between the execution overhead at all times. The tool has to carry out heavy optimizations to achieve the balance between hardware performance (latency and throughput) and implementation cost (number of LUTs, DSPs, and BRAMs) [5].

In case the HLS tool takes too long to find the theoretically perfect micro-architecture, the compiling time will go into the realm of hours, driving the software developer crazy with how slow the compilation process has become after getting used to instant compiling of the code. On the other hand, in case of too fast compiling using purely heuristics, the resulting RTL will become bloated and ineffective. The modern-day solution for this problem is providing the user with the opportunity to expose the optimization work. This can be achieved through using pragmas that artificially narrow down the search space of the SDC solver to provide an extremely optimized RTL that is human-readable as well [2].

V. Implementation of Main Features in C/C++

In order to make full use of the mathematical machinery of HLS, one needs to implement the programming concept of "hardware-aware programming." In general, conventional software languages such as C/C++ use an infinite memory assumption and von Neumann architecture, which do not exist in an FPGA itself. HLS resolves this problem through support for certain language constructs and compiler directives that define hardware micro-architectures [2], [5].

A. Language Support and Compiler Directives (Pragmas)

Modern HLS systems provide full support for all standard constructs in the C/C++ programming languages like pointers, multi-dimensional arrays, structures and IEEE-754 type of floating point data. Nevertheless, writing synthesizable code implies certain restrictions; usage of recursion and dynamic memory allocation (`malloc`/`free`) is usually forbidden as physical devices can neither be created nor destroyed dynamically in runtime [5].

The real strength of the HLS system lies in using compiler directives, which are referred to as pragmas (`#pragma HLS`). Pragmas are direct instructions embedded in the C/C++ source code that allow bypassing the default behavior of the tool and explicit specification of RTL generation rules. For example, by default an HLS tool would generate a `for` loop as a sequential state machine executing one loop iteration each clock cycle. Simply adding the directive `#pragma HLS UNROLL` allows the engineer to instruct the compiler to replicate physical hardware corresponding to the loop allowing parallel execution of multiple loop iterations [2].

In the same way, the `#pragma HLS PIPELINE` is perhaps the most important directive for high-performance DSP and stream-based applications. Through pipelining, the hardware is able to start processing a new input datum before the previous datum finishes its processing path. The efficiency of a pipeline can be measured through its initiation interval (II). When the II equals one, this implies that the FPGA is able to accept a new input datum during every clock cycle, thus increasing throughput [2]. In addition, in order to ensure that pipelined data paths are not starved of inputs, designers make use of the `#pragma HLS ARRAY_PARTITION` directive to split large C++ arrays into different Block RAMs on the FPGA [1], [5].

B. Arbitrary Precision Data Types

One of the main issues with early HLS approaches was the usage of standard C datatypes. An `int` variable occupies

32 bits, while a `char` occupies 8 bits because modern CPUs have fixed size ALUs of 32 or 64 bits. However, for FPGA designs, silicon area is an extremely expensive resource. When an algorithm needs to use an `int` variable just to store the number 1000, it will occupy 32 bits, while the same task can be performed using a 10-bit register, thus wasting 22 flip-flops and associated routing resources [1].

The solution of this problem is provided by modern HLS platforms which provide libraries that support arbitrary precision datatypes (such as `ap_int<W>`, `ap_fixed<W,I>`). Using these template classes, engineers can declare variables of any specific size. If a sensor produces 12-bit data, the engineer will declare `ap_int<12>`. The synthesis process will create exactly 12 bits wires and registers in the generated RTL netlist. This functionality alone allows the circuit to be area-efficient enough to compare with manually optimized RTL [1], [5].

C. Advancements in Task-Level Parallelism

Although loop unrolling and pipelining take care of data level parallelism, modern SoCs demand the implementation of complex and coarse-grained tasks to be executed in parallel. Traditionally, HLS faced difficulties while implementing task-parallel systems because of the serial nature of execution provided by C/C++. Recent developments have seen the creation of techniques for extending HLS to support task-parallel applications smoothly. Through the use of FIFO stream interface (`hls::stream`) of C++, different C++ functions can be synthesized in parallel [3].

VI. Realization of Application Examples

The theoretical benefits of HLS consists of reduced development time, high-level abstraction, and advanced mathematical scheduling are best demonstrated through real-world deployment case studies.

A. Case Study 1: MIMO Sphere Decoder

One of the most powerful examples of HLS success in the design process is the realization of a multi-input multi-output (MIMO) wireless system called Sphere Decoder. The Sphere Decoder is an incredibly complicated and compute-intensive algorithm used in state-of-the-art wireless communications to decode transmitted signals [1].

In this case study, the engineers employed a template-based HLS approach in order to develop a highly parallel streaming dataflow architecture. In particular, the algorithm was very parametric and allowed for using pragmas to experiment with different microarchitectures (trade-off between latency and area) without modifying the algorithm itself.

This was truly groundbreaking. Compared to a highly efficient, manually written VHDL code developed by professional hardware engineers, the automatically synthesized RTL code not only met the timing requirements but even surpassed the manual design in terms of efficiency. This allowed for reducing the number of Look-Up Tables (LUTs) by 11% and the number of registers (Flip-Flops) by 31%. The result came from the ability of the SDC algorithm to exploit the overlapping operations that the human engineers overlooked in addition to the use of arbitrary precision data types. More importantly, the result of improved QoR was achieved within a development period similar to or shorter than the RTL coding stage [1].

B. Case Study 2: Scalable Ultrafast Ultrasound Beamformer

In addition to its use in telecommunications, HLS has been demonstrated as indispensable in expediting the development of sophisticated medical devices. According to a recent publication by Kou et al. (2023), HLS has been used to design a scalable ultrafast ultrasound beamformer for a single FPGA architecture [4].

Ultrafast ultrasound imaging involves the processing of large quantities of data with the constraint of extremely low latency in order to ensure high frame rate. Conventional RTL implementation of the above stated complicated algorithm would take months of debugging. However, through writing the beamforming algorithms in C/C++ programming language and through using HLS array partitioning and pipelining pragmas, the researchers managed to deploy the above algorithm on a single FPGA. With HLS, the researchers could easily scale up the processing channels by just changing the software parameters and resynthesizing.

C. Case Study 3: The RTL Netlist Output in Practice

While case studies demonstrating system-level deployments highlight the macro-level benefits of High-Level Synthesis, it is critical to examine the micro-level output of the compilation process. The fundamental definition of HLS is its ability to bridge software and hardware; therefore, the absolute final output of the HLS process is a cycle-accurate netlist of Register-Transfer Level (RTL) components. These components are typically generated as standard VHDL entities or Verilog modules, ready for downstream logic synthesis and physical routing on the FPGA.

To concretely demonstrate this realization, we can observe the HLS translation of a fundamental DSP application: a Multiply-Accumulate (MAC) unit. The MAC unit is the mathematical backbone of both the MIMO Sphere Decoder and the Ultrafast Ultrasound Beamformer discussed previously.

In a traditional software environment, an algorithm engineer specifies the MAC intent using highly abstracted, untimed C++ code:

```
void mac_filter(int in_a, int in_b, int *out_c) {
    static int acc = 0;
    acc += in_a * in_b;
    *out_c = acc;
}
```

Upon passing this untimed code through a modern HLS compiler (such as AMD Vitis HLS), the tool applies the System of Difference Constraints (SDC) scheduling algorithms and binds the mathematical operations to physical FPGA resources. The result of this application is not a software executable, but rather a robust, synthesizable RTL component.

Below is the generated VHDL entity representing this exact C++ function:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity mac_filter is
port (
  -- Generated Control Interface
  ap_clk       : IN STD_LOGIC;
  ap_rst       : IN STD_LOGIC;
  ap_start     : IN STD_LOGIC;
  ap_done      : OUT STD_LOGIC;
  ap_idle      : OUT STD_LOGIC;
  ap_ready     : OUT STD_LOGIC;

  -- Generated Datapath
  in_a         : IN STD_LOGIC_VECTOR(31 downto 0);
  in_b         : IN STD_LOGIC_VECTOR(31 downto 0);
  out_c        : OUT STD_LOGIC_VECTOR(31 downto 0);
  out_c_ap_vld : OUT STD_LOGIC
);
end entity mac_filter;
```

Analysis of the Result: The generated VHDL netlist clearly illustrates the successful bridging of software and hardware. The HLS compiler autonomously synthesized physical hardware constraints that did not exist in the original software model. It successfully inferred a global clock (`ap_clk`) and synchronous reset (`ap_rst`). Furthermore, to ensure this hardware block can communicate synchronously with other components on the FPGA fabric, the tool automatically generated a complex block-level handshake interface containing `ap_start`, `ap_done`, `ap_idle`, and `ap_ready` signals.

The software variables `in_a` and `in_b` were successfully mapped to 32-bit standard logic vectors (`STD_LOGIC_VECTOR`), representing actual copper wires on the silicon. Ultimately, this generated VHDL entity acts as a modular, drop-in RTL component that hardware engineers can instantly simulate via testbenches or integrate into a larger System-on-Chip (SoC) architecture, proving that HLS effectively replaces the need for manual, error-prone RTL coding at the module level.

VII. Conclusion

Indeed, the direction which is being taken in terms of digital hardware design is definitely towards greater degrees of abstraction. In regards to the early behavioral compilers, which were known to have issues with predictability and audience, these limitations have definitely been surpassed thanks to the advent of High-Level Synthesis tools.

The key to success in this regard is definitely based on the use of mathematical models and the system of difference constraints (SDC). Indeed, through transforming complex C/C++ programs into a completely unimodular constraint matrix, HLS tools can make sure that integer scheduling is optimal through mathematical proof in polynomial time.

Moreover, HLS brings together the world of software and algorithms engineers by giving them direct control of hardware via pragmas and arbitrary precision datatypes. Indeed, as seen from industry case studies in fields such as telecommunications and medical imaging, progress in algorithms has made HLS very aggressive competition for traditional RTL design. In fact, today's HLS technology is not only able to compete with VHDL and Verilog in terms of Quality of Results, but is even able to beat them.

References

[1] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers and Z. Zhang, "High-Level Synthesis for FPGAs: From Prototyping to Deployment," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, April 2011, doi: 10.1109/TCAD.2011.2110592.

[2] C. Li, Y. Bi, Y. Benezeth, D. Ginhac, and F. Yang, "High-level synthesis for FPGAs: code optimization strategies for real-time image processing," *Journal of Real-Time Image Processing*, vol. 14, pp. 701–712, 2018, doi: 10.1007/s11554-017-0722-3.

[3] Y. Chi, L. Guo, J. Lau, Y. Choi, J. Wang, and J. Cong, "Extending High-Level Synthesis for Task-Parallel Programs," *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2021, pp. 204–213, doi: 10.1109/fccm51124.2021.00032.

[4] Z. Kou *et al.*, "High-Level Synthesis Design of Scalable Ultrafast Ultrasound Beamformer With Single FPGA," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 17, pp. 446–457, 2023, doi: 10.1109/TBCAS.2023.3267614.

[5] S. Lahti and T. D. Hämäläinen, "High-level Synthesis for FPGAs - A Hardware Engineer's Perspective," *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.0429000.